

SOLID

PRINCIPLES IN JAVA

A simple, detailed, example-driven guide



Design classes that are easier to understand, test, extend, and maintain.

JAVA • OBJECT-ORIENTED DESIGN • PRACTICAL EXAMPLES

1. What Does SOLID Mean?

SOLID is a group of five object-oriented design principles. They help developers organise code so that a change in one part of a program is less likely to break unrelated parts.

SOLID is not about writing more classes. It is about giving each part of the program a clear purpose and keeping dependencies under control.

Letter	Principle	Simple meaning
S	Single Responsibility	A class should have one focused responsibility.
O	Open/Closed	Add behaviour by extending code, not repeatedly editing stable code.
L	Liskov Substitution	A subtype must safely behave like the type it replaces.
I	Interface Segregation	Use small interfaces so classes implement only what they need.
D	Dependency Inversion	High-level logic should depend on abstractions, not concrete details.

Why use these principles?

- Classes become easier to understand because each class has a clearer purpose.
- New features can be added with fewer changes to working code.
- Testing becomes easier because dependencies can be replaced with test implementations.
- Code becomes less tightly coupled and more reusable.
- Large systems become safer to maintain as requirements change.

IMPORTANT

SOLID principles are guidelines, not strict laws. Use them when they reduce complexity. Do not create unnecessary interfaces or classes only to make the code look more "SOLID".

S

Single Responsibility Principle

One class, one focused responsibility

A class should have only one reason to change.

The Single Responsibility Principle does not mean that a class must contain only one method. It means that all methods in the class should support the same responsibility.

For example, a `UserValidator` may contain methods for validating a name, email address, and password. Those methods still belong to one responsibility: validating user information.

The problem: one class does everything

BAD DESIGN

```

public class Employee {
    private String name;
    private double salary;

    public double calculateYearlySalary() {
        return salary * 12;
    }

    public void saveToDatabase() {
        System.out.println("Saving employee...");
    }

    public void sendSalaryEmail() {
        System.out.println("Sending email to " + name);
    }
}

```

Why this is a problem	What can go wrong
• The class stores employee data. • It calculates salary. • It handles database access. • It sends email notifications.	• A database change modifies the <code>Employee</code> class. • An email-system change also modifies <code>Employee</code>. • The class becomes harder to test. • Unrelated responsibilities become tightly connected.

The improvement: separate responsibilities

EMPLOYEE DATA

```

public class Employee {
    private final String name;
    private final double monthlySalary;

    public Employee(String name, double monthlySalary) {
        this.name = name;
    }
}

```

```
        this.monthlySalary = monthlySalary;
    }

    public String getName() { return name; }
    public double getMonthlySalary() { return monthlySalary; }
}
```

SEPARATE SERVICES

```
public class SalaryCalculator {
    public double calculateYearlySalary(Employee employee) {
        return employee.getMonthlySalary() * 12;
    }
}

public class EmployeeRepository {
    public void save(Employee employee) {
        System.out.println("Saving " + employee.getName());
    }
}

public class EmailService {
    public void sendSalaryEmail(Employee employee) {
        System.out.println("Email sent to " + employee.getName());
    }
}
```

Now each class has one clear job. A database change affects EmployeeRepository, while an email change affects EmailService. The Employee model does not need to change for either reason.

ASK YOURSELF	MAIN BENEFIT	WARNING SIGN
Does this class have more than one unrelated job?	Smaller, focused classes that are easier to change and test.	A class changes for database, UI, calculation, and messaging reasons.



Open/Closed Principle

Open for extension, closed for modification

Software entities should be open for extension but closed for modification.

A well-designed class allows new behaviour to be added without repeatedly editing old, working logic. Interfaces and polymorphism are common tools for achieving this.

The problem: growing if/else logic

BAD DESIGN

```
public class DiscountCalculator {
    public double calculate(String customerType, double price) {
        if (customerType.equals("REGULAR")) {
            return price * 0.10;
        } else if (customerType.equals("PREMIUM")) {
            return price * 0.20;
        }
        return 0;
    }
}
```

Every new customer type requires editing the calculator. As the number of cases grows, the method becomes longer, riskier, and harder to test.

The improvement: use a strategy interface

DISCOUNT STRATEGIES

```
public interface DiscountStrategy {
    double calculateDiscount(double price);
}

public class RegularDiscount implements DiscountStrategy {
    public double calculateDiscount(double price) {
        return price * 0.10;
    }
}

public class PremiumDiscount implements DiscountStrategy {
    public double calculateDiscount(double price) {
        return price * 0.20;
    }
}
```

STABLE CALCULATOR

```
public class DiscountCalculator {
    public double calculate(
        double price,
        DiscountStrategy strategy) {
        return strategy.calculateDiscount(price);
    }
}
```

A new VIP discount can be added as a new class. The existing calculator and existing discount classes remain unchanged.

NEW EXTENSION

```
public class VipDiscount implements DiscountStrategy {  
    public double calculateDiscount(double price) {  
        return price * 0.30;  
    }  
}
```

OCP IN PRACTICE

Do not try to predict every possible future requirement. Apply OCP around parts of the system that actually change often, such as payment methods, notification channels, report formats, or pricing rules.

ASK YOURSELF

Can I add a new type without changing a large conditional?

MAIN BENEFIT

Safer extensions with less risk to existing behaviour.

WARNING SIGN

Every new case requires editing the same switch or if/else chain.

L

Liskov Substitution Principle

Subtypes must honour the promises of their parent type

An object of a subtype should be usable wherever its parent type is expected without breaking correct behaviour.

Inheritance is not correct only because two objects have an "is-a" relationship in real life. The child class must also support the behaviour promised by the parent class.

The problem: a penguin that is forced to fly

BAD INHERITANCE

```

public class Bird {
    public void fly() {
        System.out.println("Bird is flying");
    }
}

public class Penguin extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException(
            "Penguins cannot fly"
        );
    }
}

```

A method accepting Bird reasonably expects fly() to work. Passing Penguin breaks that expectation, so Penguin is not a safe substitute for Bird as designed.

The improvement: model capabilities separately

CAPABILITY-BASED DESIGN

```

public abstract class Bird {
    public abstract void eat();
}

public interface Flyable {
    void fly();
}

public class Sparrow extends Bird implements Flyable {
    public void eat() {
        System.out.println("Sparrow is eating");
    }

    public void fly() {
        System.out.println("Sparrow is flying");
    }
}

public class Penguin extends Bird {
    public void eat() {
        System.out.println("Penguin is eating");
    }

    public void swim() {

```

```

        System.out.println("Penguin is swimming");
    }
}

```

Only birds that can actually fly implement Flyable. Penguin is no longer forced to provide a method that throws an exception.

Another common example: Rectangle and Square

A mutable Rectangle usually allows width and height to change independently. A Square cannot preserve that behaviour because changing one side must change the other. Therefore, a mutable Square may not safely replace a mutable Rectangle.

SAFER DESIGN

```

public interface Shape {
    int getArea();
}

public final class Rectangle implements Shape {
    private final int width;
    private final int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    public int getArea() {
        return width * height;
    }
}

public final class Square implements Shape {
    private final int side;

    public Square(int side) {
        this.side = side;
    }

    public int getArea() {
        return side * side;
    }
}

```

ASK YOURSELF

Can every implementation safely satisfy the parent contract?

MAIN BENEFIT

Predictable polymorphism without surprising exceptions or behaviour.

WARNING SIGN

A child overrides a method only to throw "unsupported" or changes its meaning.

I

Interface Segregation Principle

Prefer small, focused interfaces

A class should not be forced to depend on methods that it does not use.

Large interfaces are sometimes called “fat interfaces”. They force implementing classes to provide methods that may not make sense for them.

The problem: one interface for every worker

BAD DESIGN

```
public interface Worker {
    void work();
    void eat();
    void sleep();
}

public class RobotWorker implements Worker {
    public void work() {
        System.out.println("Robot is working");
    }

    public void eat() {
        throw new UnsupportedOperationException();
    }

    public void sleep() {
        throw new UnsupportedOperationException();
    }
}
```

RobotWorker is forced to implement eat() and sleep(), even though those behaviours do not belong to a robot.

The improvement: split the interface by capability

SMALL INTERFACES

```
public interface Workable {
    void work();
}

public interface Eatable {
    void eat();
}

public interface Sleepable {
    void sleep();
}
```

ONLY IMPLEMENT WHAT IS NEEDED

```
public class HumanWorker
    implements Workable, Eatable, Sleepable {

    public void work() { }
```

```
    public void eat() { }  
    public void sleep() { }  
}  
  
public class RobotWorker implements Workable {  
    public void work() { }  
}
```

The robot now implements only `Workable`. The human can combine all three interfaces.

JAVA EXAMPLE

The standard Java interfaces `Runnable`, `Comparable<T>`, `AutoCloseable`, and `Iterable<T>` are focused interfaces. A class can implement only the capabilities it needs.

ASK YOURSELF

Does this class implement methods it cannot meaningfully support?

MAIN BENEFIT

Simpler contracts and fewer dummy or exception-throwing methods.

WARNING SIGN

Methods are empty, return meaningless values, or throw unsupported exceptions.

D

Dependency Inversion Principle

Depend on abstractions, not concrete implementations

High-level modules should not depend directly on low-level modules. Both should depend on abstractions.

High-level code represents business rules. Low-level code handles technical details such as email, files, databases, or external APIs. Business logic should not be tightly attached to one technical implementation.

The problem: creating a concrete dependency inside the class

TIGHT COUPLING

```

public class EmailService {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}

public class OrderService {
    private final EmailService emailService =
        new EmailService();

    public void placeOrder() {
        System.out.println("Order placed");
        emailService.send("Order completed");
    }
}

```

OrderService is tied to EmailService. Switching to SMS or testing the order logic without sending a real email requires modifying the class.

The improvement: depend on an interface

ABSTRACTION AND IMPLEMENTATIONS

```

public interface NotificationService {
    void send(String message);
}

public class EmailNotificationService
    implements NotificationService {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}

public class SmsNotificationService
    implements NotificationService {
    public void send(String message) {
        System.out.println("SMS: " + message);
    }
}

```

CONSTRUCTOR INJECTION

```

public class OrderService {
    private final NotificationService notificationService;

    public OrderService(
        NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    public void placeOrder() {
        System.out.println("Order placed");
        notificationService.send("Order completed");
    }
}

```

OrderService now depends on NotificationService, not on email or SMS. The required implementation is supplied from outside the class.

Dependency Inversion vs Dependency Injection

Dependency Inversion Principle	Dependency Injection
• A design principle. • Says to depend on abstractions. • Reduces coupling between policy and implementation.	• A technique. • Provides dependencies from outside a class. • Constructor injection is usually the clearest Java option.

SPRING CONNECTION

Spring commonly creates objects and injects their dependencies. However, Dependency Injection can be used in plain Java without any framework.

ASK YOURSELF Does my business class create a database, email, or API implementation itself?	MAIN BENEFIT Flexible, replaceable dependencies and easier unit testing.	WARNING SIGN The new keyword appears inside business logic for infrastructure services.
---	--	---

7. Complete Example: A SOLID Payment System

The following example combines multiple SOLID principles. The payment service coordinates the operation while payment and receipt details are represented by focused abstractions.

FOCUSED ABSTRACTIONS

```
public interface PaymentMethod {
    void pay(double amount);
}

public interface ReceiptService {
    void sendReceipt(double amount);
}
```

IMPLEMENTATIONS

```
public class CreditCardPayment implements PaymentMethod {
    public void pay(double amount) {
        System.out.println(
            "Paid " + amount + " using credit card"
        );
    }
}

public class PayPalPayment implements PaymentMethod {
    public void pay(double amount) {
        System.out.println(
            "Paid " + amount + " using PayPal"
        );
    }
}

public class EmailReceiptService implements ReceiptService {
    public void sendReceipt(double amount) {
        System.out.println(
            "Receipt sent for " + amount
        );
    }
}
```

HIGH-LEVEL SERVICE

```
public class PaymentService {
    private final PaymentMethod paymentMethod;
    private final ReceiptService receiptService;

    public PaymentService(
        PaymentMethod paymentMethod,
        ReceiptService receiptService) {
        this.paymentMethod = paymentMethod;
        this.receiptService = receiptService;
    }

    public void processPayment(double amount) {
        paymentMethod.pay(amount);
        receiptService.sendReceipt(amount);
    }
}
```

USAGE

```
public class Main {
    public static void main(String[] args) {
        PaymentMethod payment =
```

```
        new CreditCardPayment();

    ReceiptService receipt =
        new EmailReceiptService();

    PaymentService service =
        new PaymentService(payment, receipt);

    service.processPayment(500.0);
}
```

Where the five principles appear

SRP

Each implementation has one focused responsibility. `PaymentService` coordinates the use case; it does not contain card-processing or email details.

OCP

A `CashPayment` or `BankTransferPayment` class can be added without modifying `PaymentService`.

LSP

Any valid `PaymentMethod` implementation can be supplied wherever `PaymentMethod` is expected.

ISP

`PaymentMethod` and `ReceiptService` are small interfaces. Payment classes are not forced to send receipts.

DIP

`PaymentService` depends on `PaymentMethod` and `ReceiptService` abstractions rather than concrete classes.

8. Why SOLID Makes Testing Easier

When a class depends on an interface, a small fake implementation can be supplied during a test. This keeps the test fast and avoids real email, database, or network operations.

TEST DOUBLE

```
public class FakeNotificationService
    implements NotificationService {

    private String lastMessage;

    public void send(String message) {
        this.lastMessage = message;
    }

    public String getLastMessage() {
        return lastMessage;
    }
}
```

SIMPLE TEST

```
public class OrderServiceTest {
    public static void main(String[] args) {
        FakeNotificationService fake =
            new FakeNotificationService();

        OrderService service = new OrderService(fake);
        service.placeOrder();

        boolean correct = "Order completed"
            .equals(fake.getLastMessage());

        System.out.println(correct); // true
    }
}
```

OrderService can be tested without connecting to an email server. This is a practical result of depending on an abstraction and injecting the dependency.

Composition is often better than inheritance

Many SOLID designs prefer composition: an object receives other objects that provide specific behaviour. Composition is usually more flexible than building a deep inheritance hierarchy.

Inheritance	Composition
• Represents an “is-a” relationship. • Can tightly connect parent and child behaviour. • Incorrect contracts may violate LSP.	• Represents a “has-a” or “uses-a” relationship. • Behaviour can be replaced at runtime or during tests. • Works naturally with interfaces and Dependency Injection.

9. SOLID Cheat Sheet

Use these questions while reviewing a class or designing a new feature.

	Principle	Ask yourself	Common warning sign
S	Single Responsibility	Does this class have one focused job?	The class changes for several unrelated reasons.
O	Open/Closed	Can I add a new case without editing stable logic?	A switch or if/else grows with every new type.
L	Liskov Substitution	Can every subtype honour the same contract?	A subtype throws unsupported exceptions or changes expectations.
I	Interface Segregation	Does every implementer need every method?	Classes contain empty or meaningless methods.
D	Dependency Inversion	Does business logic depend on an abstraction?	Business code creates concrete infrastructure services itself.

A practical decision process

1. Start with the simplest design that clearly solves the current requirement.
2. Identify responsibilities that change for different reasons and separate them when necessary.
3. Use interfaces around behaviour that has multiple implementations or must be replaced during testing.
4. Check that every implementation honours the same behavioural contract.
5. Keep interfaces focused and inject infrastructure dependencies into business services.
6. Refactor when duplication, large conditionals, tight coupling, or unsupported methods appear.

FINAL REMINDER

SOLID is successful when the resulting code is easier to read and change. If applying a principle creates more complexity than it removes, simplify the design.

S O L I D

Focused responsibilities • Safe extensions • Reliable subtypes • Small interfaces • Abstract dependencies